Lab 4 Konto firmowe, coverage i opis PR

1 Cel zajęć

Dokładamy do projektu konto firmowe, mierzymy, ile kodu pokrywają testy, i uczymy agenta AI pisać opis pull requesta według Twojego szablonu.

- **Po zajęciach:** wydzielisz wspólny kod kont do klasy bazowej, mając testy jako siatkę bezpieczeństwa,
- zmierzysz coverage lokalnie i sprawdzisz raport HTML, a CI będzie czerwone, gdy pokrycie spadnie poniżej 100%,
- dopiszesz szablon opisu PR w AGENTS.md i użyjesz go do przygotowania PR.

Kod, nazwy testów, commity i prompty piszemy **po angielsku**.

2 Start: Lab 3 w main

> Praca z Lab 3 musi być zmergowana do `main` (PR zamknięty, CI zielone). Jeśli nie jest, dokończ PR. Potem:

```
git switch main
git pull
python -m pytest
git switch -c lab4
```

3 Konto firmowe

Konto firmowe ma nazwę firmy i NIP zamiast imienia, nazwiska i PESEL-u. Utwórz `CompanyAccount(company_name, nip)` w `src/company_account.py`. NIP ma dokładnie 10 cyfr, inaczej pole `nip` ma wartość `"Invalid"` (tak jak PESEL; sumy kontrolnej nie sprawdzamy). Saldo startuje od 0, kodu promocyjnego nie ma. Wpłata i wypłata działają jak w `Account` (te same wyjątki).

1. **Red.** Napisz testy: poprawny NIP, za krótki, za długi, z literą, pusty, wpłata, wypłata i oba błędy wypłaty.
2. **Green.** Zaimplementuj klasę. **Możesz skopiować** `deposit` i `withdraw` z `Account`. Zrobimy z tym porządek za chwilę.
3. **Refactor.** Dwie klasy mają ten sam kod. Zastanów się, jak najlepiej zrobić refaktor, żeby wpłata i wypłata były w jednym miejscu. Testy zostają bez zmian i mają być zielone.

Uwaga: rejestr kont obsługuje na razie tylko konta osobiste (kluczem jest PESEL). Konto firmowe wejdzie do rejestru na kolejnych zajęciach.

> **Zrobione, gdy:**
> - testy CompanyAccount i wszystkie dotychczasowe testy zielone,
> - kod wpłaty i wypłaty nie jest zduplikowany,
> - osobne commity: testy, implementacja, refaktor

4 Coverage: ile kodu sprawdzają testy

Coverage to procent linii kodu, które wykonały się podczas testów. Niskie pokrycie oznacza kod, którego nikt nie sprawdza. Wysokie nie dowodzi, że testy są dobre (nie wiemy, czy cokolwiek sprawdzają), ale pozwala znaleźć luki.

4.1 Lokalnie

Dopisz `coverage` do `requirements.txt`, zainstaluj i uruchom:

```
python -m pip install -r requirements.txt
python -m coverage run --source=src -m pytest
python -m coverage report --show-missing    # column Missing = lines without tests
python -m coverage html                     # open htmlcov/index.html in a browser
```

W raporcie HTML kliknij plik z kodem: linie niepokryte są zaznaczone na czerwono. Jeśli pokrycie jest poniżej 100%, dopisz brakujące testy.

4.2 W CI

Zmień `.github/workflows/tests.yml`, żeby testy szły przez `coverage run`, a po nich był krok z raportem w terminalu, który **kończy się błędem, gdy pokrycie jest poniżej 100%**. Szukaj odpowiedniej opcji w dokumentacji coverage. Sprawdź to: dopisz metodę bez testu, wypchnij, zobacz czerwone CI, potem usuń metodę.

> **Zrobione, gdy:**
>
> - lokalnie 100% i obejrzany raport HTML,
> - CI robi pomiar i jest czerwone poniżej 100% (sprawdzone na metodzie bez testu),
> - commit

5 Opis PR przygotowany przez agenta

W `AGENTS.md` dopisz sekcję z szablonem opisu PR. Agent będzie go używał za każdym razem, więc opisy będą spójne. Szablon wklej w całości, razem z liniami z trzema apostrofami (```), żeby jego nagłówki nie stały się sekcjami AGENTS.md:

````
```
## Pull request description
When asked for a pull request description, base it on `git diff main` and use
exactly this template (in English):

## Summary
One or two sentences: what changed and why.
## Changes
- bullet list of the changes
## Testing
- how it was tested (tests added, commands run, coverage)

Describe what changed. Do not review or grade the code in the description.
```
````

1. Uruchom `claude` lub `gemini` w folderze projektu i napisz: `Write a pull request description for the changes on the lab4 branch compared to main.`
2. Sprawdź opis: czy zgadza się z tym, co faktycznie zrobiłeś, czy nic nie zmyślił, czy trzyma szablon. Popraw go.
3. Skopiuj opis, utwórz na GitHubie PR lab4 → main i wklej go.

Zrobione, gdy:

- AGENTS.md ma szablon opisu PR,
- PR lab4 → main jest otwarty z opisem w szablonie, sprawdzonym i poprawionym,
- CI zielone (100% coverage), merge zrobiony

Dodatkowo: agent sam tworzy PR (na własnym komputerze)

Uwaga: token daje dostęp do Twojego konta GitHub. Nie wpisuj go na komputerach uczelnianych, nie commituj go i nadaj tylko niezbędne uprawnienia oraz krótki czas ważności.

- **GitHub CLI** (bez własnego tokenu): po instalacji i `gh auth login` agent może wywołać `gh pr create`. Zob. gh pr create.
- **Serwer MCP GitHuba:** wymaga tokenu (PAT). repozytorium github-mcp-server (instrukcje dla Gemini CLI w docs/installation-guides), MCP w Claude Code, tworzenie tokenu.

Reguła w AGENTS.md zabrania agentowi otwierania PR-ów. Jeśli chcesz to zmienić, zmień ją świadomie i dopisz, że agent ma pokazać opis do akceptacji przed utworzeniem PR.

6 Zadanie domowe: fixtures

Testy z Lab 3 powtarzają tworzenie rejestru i kont. **Fixture** to funkcja, która przygotowuje dane dla testu. Pytest podaje jej wynik jako argument:

```
import pytest

@pytest.fixture
def registry():
    return AccountRegistry()

def test_new_registry_is_empty(registry):    # pytest passes the fixture result here
    assert registry.count() == 0
```

Zamień powtarzające się przygotowanie danych w testach rejestru i kont na fixtures. Zadanie zrób na nowej gałęzi, w osobnym commicie i z PR-em. Zob. pytest: fixtures.

Zrobione, gdy:

- powtórzenia zastąpione fixtures, liczba testów bez zmian,
- pytest zielony, coverage 100%, PR i merge

7 Gdy coś nie działa

- `No data to report`: najpierw `coverage run`, dopiero potem `coverage report`.
- CI czerwone, lokalnie zielone: sprawdź, czy `coverage` jest w wypchniętym `requirements.txt` i czy plik workflow jest na wypychanej gałęzi.
- Nie masz `gh` lub MCP: nie szkodzi, opis PR kopiujesz ręcznie. To pełnoprawna ścieżka zadania.

Do poczytania: coverage.py · dziedziczenie w Pythonie.